

第8.2课

《智能合约安全基础》



安全防护与漏洞分析



深入理解以太坊智能合约安全问题

从原理到防御，全方位掌握安全技能

目录

智能合约安全基础课程内容概览

🔥 6大安全漏洞

- 🔄 重入攻击 (Reentrancy)
- ⊕ 整数溢出 (Integer Overflow)
- 🔒 权限控制 (Permission Control)
- 🚫 拒绝服务 (DoS Attack)
- 🏃 前端运行攻击 (Front-Running)
- 🛡️ 安全实践与最佳实践 🔒

⚠️ 真实案例损失

\$6000万

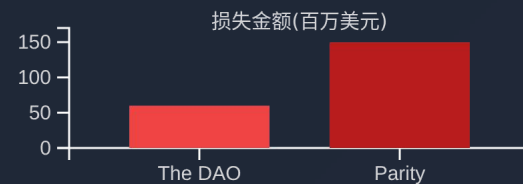
The DAO事件损失

🕒 2016年6月

\$1.5亿

Parity多签钱包冻结

2017年7月



重入攻击原理

Reentrancy Attack Principles

什么是重入攻击

定义：攻击者通过恶意合约，多次调用同一函数，从而在预期操作完成前获取不当利益。

核心问题：合约未正确管理状态，允许在交易执行过程中被重新进入。

危险等级： 高危

攻击流程

1 外部调用

攻击者合约调用目标合约的提取函数

2 恶意回调

恶意合约在fallback函数中递归调用

3 状态更新

目标合约更新状态

4 重复提取

攻击者可重复提取资金

不安全代码示例

```
// 不安全的重入攻击示例  
function withdraw(uint256 amount) public {
```

```
    uint256 balance = balances[msg.sender];  
    require(balance >= amount);  
    msg.sender.call.value(amount)();
```

```
    balances[msg.sender] = balance - amount;
```

```
}
```

```
function fallback() public payable {
```

```
    if (address(this).balance >= 1 ether) {
```

```
        withdraw(1 ether);
```

```
    }
```

```
}
```

💡 先转账后检查余额，导致可重复调用

The DAO案例分析

2016年6月，\$6000万以太币被盗事件

\$ 60M

损失金额

以太币价值约6000万美元

</> 漏洞代码分析

vulnerable code

```
function withdraw(uint256 amount) {
    require(balances[msg.sender] >= amount);

    // 外部调用在状态更新之前
    msg.sender.call.value(amount)(); // <--- 漏洞点

    // 状态更新在外部调用之后
    balances[msg.sender] -= amount; // <--- 可被重入攻击利用
}
```

❗ 在外部调用 (call.value

重入攻击防御方法

三种有效防御策略对比与实践

CEI模式

```
// 1. 检查状态
function withdraw(uint amount) public {
    require(balances[msg.sender] >= amount);
    // 2. 更新状态
    balances[msg.sender] -= amount;
    totalSupply -= amount;
    // 3. 外部调用
    payable(msg.sender).transfer(amount);
}
```

- + 优点：简单易实现
- 缺点：复杂逻辑下难以维护

重入锁

```
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract Vault is ReentrancyGuard {
    function withdraw(uint amount)
    public nonReentrant {
        // 无需显式检查，编译器自动插入
        payable(msg.sender).transfer(amount);
        balances[msg.sender] -= amount;
        totalSupply -= amount;
    }
}
```

- + 优点：自动化检查
- 缺点：增加Gas消耗

OpenZeppelin最佳实践

- ✓ Ownable
- ✓ AccessControl
- ✓ ReentrancyGuard
- ✓ SafeMath

三种方法对比

防御方法	实现难度	Gas消耗	维护成本
CEI模式	★★★☆☆	★★★☆☆	★★★☆☆
重入锁	★★★★☆	★★★★☆	★★★★☆

整数溢出原理和危害

当数值超出数据类型最大值时发生的问题

整数溢出可视化



❗ 无符号整数达到最大值后溢出回到0

❗ 有符号整数溢出导致行为异常

0.7.6版本漏洞演示

不安全代码

```
// Solidity 0.7.6 中的整数溢出漏洞
function safeSub(uint256 a, uint256 b) internal pure returns
(uint256) {
    require(b <= a, "Subtraction overflow");
    → return a - b; // 编译器不会检测此行是否溢出
}
```

⚠ 在0.7.6中，编译器不会自动检查溢出，需开发者手动添加检查

BeautyChain真实案例

漏洞类型

整数溢出导致奖励分配错误

影响

用户无法提取奖励，资金被困

🔑 攻击者通过特定交易触发溢出，获取不当收益

Solidity 0.8.0保护机制

版本升级与安全特性

版本对比 (0.7.6 vs 0.8.0)

</> 0.7.6

默认不进行溢出检查

```
// pragma solidity ^0.7.6;
function add(uint256 a,
uint256 b) public pure
returns (uint256) {
return a + b; // 可能溢出但不报错
}
```

Unchecked块说明

0.8.0引入unchecked块允许开发者手动控制溢出检查：

```
function safeAdd(uint256 a, uint256 b) public pure returns
(uint256) {
unchecked {
return a + b; // 禁用溢出检查
}
}
```

使用0.8.0+版本可有效防止整数溢出漏洞

0.8.0

默认进行溢出检查

```
// pragma solidity ^0.8.0;
function add(uint256 a,
uint256 b) public pure
returns (uint256) {
return a + b; // 自动检查溢出,
超出时panics
}
```

Gas优化技巧

- 使用unchecked块减少Gas消耗
- 明确变量类型避免不必要的转换
- 合理使用require和revert

最佳实践

- 1 优先使用0.8.0+版本
- 2 对关键计算使用unchecked块

Gas优化与安全性平衡

权限控制漏洞

缺少权限检查的危害与防御

⚠️ 缺少权限检查的危害

- ☠️ 任何人可修改关键状态变量
- 🏴‍☠️ 恶意用户可执行特权操作

🚑 常见权限漏洞类型

- 1 **missing access modifiers** – 缺少访问修饰符
- 2 **improper caller checks** – 不正确的调用者检查
- 3 **delegatecall risks** – 委托调用风险

</> 不安全vs安全代码对比

❌ 不安全代码

```
contract VulnerableToken {
    address public owner;
    mapping(address => uint256) public balances;

    // 缺少权限检查
    function transfer(address to, uint256 amount) public {
        balances[msg.sender] -= amount;
        balances[to] += amount;
    }

    // 可能被任何人调用，导致owner被修改
    function changeOwner(address newOwner) public {
        owner = newOwner;
    }
}
```

✅ 安全代码

```
contract SecureToken {
    address public owner;
    mapping(address => uint256) public balances;

    // 使用onlyOwner修饰符进行权限检查
    function transfer(address to, uint256 amount) public onlyOwner {
        balances[msg.sender] -= amount;
        balances[to] += amount;
    }

    function changeOwner(address newOwner) private {
        owner = newOwner;
    }
}
```


Parity案例分析

150百万美元冻结事件剖析

事件概述

 **\$150,000,000**
Parity多签钱包资金冻结

事件时间线

2017年7月

Parity多签钱包库合约意外开源

漏洞发现

发现initWallet函数可被任意调用

攻击利用

攻击者调用initWallet初始化钱包，导致资金被冻结

漏洞代码分析

```
function initWallet(address[] _owners, uint _threshold) {  
    require(owners.length > 0);  
    require(_threshold > 0);  
  
    // 问题：缺少权限检查  
    owners = _owners;  
    threshold = _threshold;  
  
    // 攻击者可重复调用此函数  
}
```

 **关键问题：**缺少权限检查

 **自毁问题：**库合约可被自毁

权限控制教训

 **明确权限控制**
始终检查调用者权限

 **初始化检查**
避免重复初始化

 **库合约安全**
防止意外暴露函数

 **访问控制**
谨慎设计公共函数

OpenZeppelin权限控制实践

安全的权限管理实现方案

Ownable合约

最基础的权限控制合约，只有一个所有者地址

```
import "@openzeppelin/contracts/access/Ownable.sol";

contract MyContract is Ownable {
    function doSomething() public onlyOwner {
        // 只有所有者可以调用
    }
}
```

✓ 简单直观的权限控制

⚠ 单点故障风险

AccessControl角色管理

细粒度权限控制系统，支持多个角色

```
import "@openzeppelin/contracts/access/AccessControl.sol";

contract MyContract is AccessControl {
    bytes32 public constant ADMIN_ROLE =
        keccak256("ADMIN_ROLE");
    bytes32 public constant USER_ROLE =
        keccak256("USER_ROLE");

    constructor() {
        _grantRole(ADMIN_ROLE, msg.sender);
    }

    function grantRole() public onlyRole(ADMIN_ROLE) {
        _grantRole(USER_ROLE, msg.sender);
    }
}
```

✓ 多角色权限系统

✓ 角色继承与授权

多重签名方案

需要多个地址签名才能执行操作

```
import "@openzeppelin/contracts/access/multisig/MultiSigWallet.sol";

contract Wallet is MultiSigWallet {
    address[] public owners;

    constructor(address[] memory _owners, uint _required)
        public MultiSigWallet(_owners, _required) {
        owners = _owners;
    }

    function executeTransaction(
        address to,
        uint value,
        bytes memory data
    ) public onlyOwner {
        // 只有所有者可以执行
    }
}
```

✓ 提高资金安全级别

⚠ 执行速度较慢

DoS攻击类型

拒绝服务攻击的四种主要形式



Gas耗尽攻击

攻击者通过消耗大量Gas来阻止交易执行

- 递归调用消耗过多Gas
- 大量状态修改操作



状态阻塞攻击

通过控制状态变量导致其他用户无法操作

- 独占关键状态变量
- 修改访问控制状态



外部依赖攻击

利用外部合约调用失败导致交易回滚

- 不可信外部合约调用
- 外部API响应异常



循环阻塞攻击

通过恶意输入导致循环执行过多次

- 不合理的循环次数
- 嵌套循环结构风险

所有DoS攻击都会导致交易无法执行或消耗过多Gas，增加交易成本

Gas耗尽攻击演示

Push Model缺陷与奖励分配安全问题

⚠️ 推送模式问题

🚫 Gas限制

以太坊交易Gas上限为672,197

⚡ 成本递增

奖励数量增加导致Gas消耗呈二次方增长

🚫 执行中断

任何接收者失败都会导致整个交易回滚

</> 不安全奖励分配

```
pragma solidity ^0.7.6;
contract RewardDistributor {
    address owner;
    mapping(address => uint256) rewards;
    uint256 totalRewards;

    function distributeRewards(address[]
_recipients) public {
        require(msg.sender == owner, "Not
owner");
        for(uint256 i=0;
i<_recipients.length; i++) {
            rewards[_recipients[i]] +=
totalRewards / _recipients.length;
        }
    }

    function claimReward() public {
        require(rewards[msg.sender] > 0, "No
reward");
        uint256 amount = rewards[msg.sender];

        rewards[msg.sender] = 0;
        require(address(this).balance >=
amount, "Insufficient balance");
        payTo(msg.sender, amount);
    }

    function payTo(address to,
```

💀 攻击场景

👤 攻击者部署恶意合约

实现接收ether的fallback函数，消耗Gas

🔗 调用奖励分配函数

传入大量恶意地址，使Gas消耗超过交易上限

❌ 交易回滚

所有奖励分配失败，攻击者获得所有ether

拉取模式防御

Pull over Push 原则与分页处理方案

↔ Pull over Push原则

推送模式问题

- Gas耗尽风险
- 交易失败导致状态不一致
- 奖励分配不均

拉取模式优势

- 接收方控制Gas使用
- 可分批处理交易
- 提高交易成功率



</> 代码实现

❌ 推送模式（不安全）

```
// 不安全的奖励分配
contract PushMode {
    address[] contributors;
    mapping(address => uint256) rewards;
    function distributeRewards() public {
        for (uint256 i = 0; i < contributors.length; i++) {
            contributors[i].call.value(rewards[contributors[i]]())();
        }
    }
}
```

✅ 拉取模式（安全）

```
// 安全的拉取模式
contract PullMode {
    mapping(address => uint256) rewards;
    function requestReward() public {
        uint256 amount = rewards[msg.sender];
        if (amount > 0) {
            rewards[msg.sender] = 0;
            msg.sender.transfer(amount);
        }
    }
}
```

📄 分页处理方案

🔍 分页原理

- 限制每批次处理的项目数量
- 逐步处理大型数据集
- 避免单次交易Gas过度消耗

</> 分页代码示例

```
// 分页处理示例
function processBatch(uint256 start, uint256 limit) public view returns (uint256 processed) {
    for (uint256 i = start; i < start + limit; i++) {
        processed++;
    }
}
```

☰ 分页导航



前端运行攻击原理

什么是Front-Running及Mempool攻击流程

什么是前端运行攻击

定义

前端运行攻击（Front-Running）是指攻击者观察到某笔交易在区块链网络中，**抢先提交自己的交易**，以影响原交易的执行结果或获取不当利益。

攻击特点

- 利用交易传播延迟
- 抢先执行相似操作
- 通常涉及DEX交易
- 可获取不当利润

Mempool攻击流程



攻击原理：攻击者抢先提交更高Gas价格的交易，获取交易差价。

MEV概念介绍

MEV是什么？

MEV (Maximal Extractable Value) 表示通过交易排序和替换所能获取的最大价值总和，是区块链经济中的重要概念。

Flashbots解决方案

- 私有交易池 (Private Transaction Pool)
- 提高交易隐私性
- 防止交易观察和抢跑
- 优化矿工收益

攻击场景示例

在DEX交易中，攻击者观察到一笔大额交易，**抢先提交更高Gas价格的交易**，从而获取交易差价。

Commit-Reveal防御方案

防止前端运行攻击的两阶段提交机制

两阶段提交原理

第一阶段：Commit（提交）

- 用户提交交易哈希值
- 锁定资金或状态

第二阶段：Reveal（揭示）

- 公布原始秘密值
- 验证哈希匹配并执行操作

Commit/Reveal代码

Commit函数

```
function commit(bytes32 secretHash) public {
    require(commitments[msg.sender] == 0, "Already committed");
    commitments[msg.sender] = secretHash;
    emit Committed(msg.sender, secretHash);
}
```

Reveal函数

```
function reveal(bytes32 secret) public {
    require(commitments[msg.sender] != 0, "Not committed");
    require(keccak256(abi.encodePacked(secret)) == commitments[msg.sender], "Hash mismatch");
    // 执行操作...
    delete commitments[msg.sender];
    emit Revealed(msg.sender, secret);
}
```

投票合约示例

```
contract VoteCounter {
    mapping(address => bytes32) public commitments;
    mapping(bytes32 => uint) public votes;

    function commitVote(bytes32 voteHash) public {
        require(commitments[msg.sender] == 0, "Already voted");
        commitments[msg.sender] = voteHash;
        emit Committed(msg.sender, voteHash);
    }

    function revealVote(bytes32 vote, uint salt) public {
        require(commitments[msg.sender] != 0, "Not committed");
        require(keccak256(abi.encodePacked(vote, salt)) == commitments[msg.sender], "Invalid vote");
        votes[vote]++;
        delete commitments[msg.sender];
        emit VoteRevealed(msg.sender, vote);
    }

    function getVoteCount(bytes32 vote) public view returns (uint) {
        return votes[vote];
    }

    event Committed(address indexed voter, bytes32 indexed hash);
    event VoteRevealed(address indexed voter, bytes32 vote);
}
```

前端运行其他防御方法

防止前端运行攻击的多种技术方案



Flashbots私有交易池

- ✓ 将交易打包为一个整体提交
- ✓ 隐藏交易内容和顺序
- ✓ 防止矿工提取价值

适用于: DeFi协议、交易所



价格滑点限制

- ✓ 设置最大价格偏离阈值
- ✓ 自动调整交易参数
- ✓ 防止恶意价格操纵

适用于: DEX、AMM



批量拍卖机制

- ✓ 收集多个交易批量处理
- ✓ 统一计算gas价格
- ✓ 平滑交易成本

适用于: 交易所、预言机



最小延迟策略

- ✓ 引入随机延迟提交
- ✓ 增加价格预测难度
- ✓ 结合Commit-Reveal方案

适用于: 投票、预言机

 **最佳实践:** 结合多种防御方法，根据应用场景选择合适的技术方案，平衡用户体验和安全性。

安全检查清单

智能合约开发安全检查项

🔄 重入防护

- ✓ 使用CEI模式（检查-交互-修改）
- ✓ 实施重入锁（ReentrancyGuard）
- ✓ 优先使用pull模式而非push模式
- ✓ 敏感操作前重置状态变量
- ✓ 避免外部调用时修改状态
- ✓ 使用OpenZeppelin的防护库

🔒 权限控制

- ✓ 验证msg.sender权限
- ✓ 使用onlyOwner修饰符
- ✓ 角色基于角色管理（AccessControl）
- ✓ 谨慎处理权限转移
- ✓ 避免使用tx.origin进行权限检查
- ✓ 实现多重签名方案

🛡️ 输入验证

- ✓ 验证所有外部输入
- ✓ 检查数组索引越界
- ✓ 限制输入范围和长度
- ✓ 使用require进行前置条件检查
- ✓ 避免unchecked_call返回值
- ✓ 使用SOLIDITY版本0.8.0+

📋 其他安全检查

- ✓ 检查整数溢出/下溢
- ✓ 避免Gas耗尽风险
- ✓ 防范前端运行攻击
- ✓ 状态变量封装
- ✓ 使用unchecked块优化Gas
- ✓ 定期安全审计

💡 安全检查清单应作为开发流程的一部分，确保代码审查全面性

安全开发流程

构建安全智能合约的全流程方法



安全开发循环: 安全不是一次性工作，而是贯穿整个开发周期的过程。定期培训团队，保持对新漏洞的关注，并不断完善安全流程。

课程总结

5种漏洞对比 | 核心防御方法 | 关键点

5种漏洞对比表

漏洞类型	危险等级	主要影响	防御方法
重入攻击	高危	资金损失	CEI模式、重入锁
整数溢出	中危	数值异常	0.8.0版本检查
权限控制	高危	未授权操作	AccessControl
拒绝服务	中危	交易失败	拉取模式
前端运行	中危	交易被篡改	Commit-Reveal

核心防御方法

- CEI模式

检查-交互-修改顺序
- 权限控制

Ownable/AccessControl
- 输入验证

白名单过滤
- 重入锁

ReentrancyGuard
- 拉取模式

Pull over Push
- Gas限制

安全阈值检查

关键点回顾

- ✔ 优先使用成熟开源库如OpenZeppelin
- ✔ 避免外部调用，尤其是不受信任的合约
- ✔ 始终验证输入参数和状态变量
- ✔ 使用unchecked块明确处理溢出风险
- ✔ 采用两阶段提交防止前端运行攻击
- ✔ 定期安全审计和代码审查

实践作业

巩固所学知识，提升安全开发能力

作业1：修复重入漏洞 ★★★★

修复给定合约中的重入攻击漏洞，确保资金安全

 Solidity, 安全修复

作业2：实现权限控制 ★★★★

设计并实现一个权限控制系统，防止权限滥用

 AccessControl, Ownable

作业3：防御DoS攻击 ★★★★

实现拉取模式而非推送模式，防御Gas耗尽攻击

 Pull over Push, 分页处理

作业4：Commit-Reveal投票 ★★★★☆

实现两阶段提交机制，防止前端运行攻击

 Commit-Reveal, 隐私保护

作业5：安全审查实战 ★★★★

对复杂合约进行安全审计，找出潜在漏洞并提出修复方案

 审计报告, 漏洞利用

额外挑战



Ethernaut挑战

完成Ethernaut合约靶场中的漏洞挑战，提升实战能力



真实案例研究

分析最近的智能合约安全事件，撰写详细复盘报告

 提示：所有作业将提交至课程平台，优秀作品将在课程总结中展示